

Projeto ToDo List: *tarefas concluídas*

A segunda versão do ToDo List ganha o que toda lista de tarefas precisa: **marcar como concluída**, ver o **progresso**, **filtrar** pendentes e concluídas e **limpar** o que já foi feito. E aparece um bug sutil, do tipo que o agente deixa passar, para você aprender a pegar e corrigir.

Dicionário por tarefa

`st.checkbox`

`st.progress`

Filtro com `st.radio`

Chaves estáveis

Teste de regressão

Formação Claude Code 2026 · Baseado em cursor.com/docs (Agent, Plan Mode, Browser) e docs.streamlit.io (`st.checkbox`, `st.progress`, `st.radio`, Widget keys), consultados em outubro de 2026 (Streamlit 1.65) · Código de referência testado em `projeto/app.py` · Telas: print real

Da versão 1 *para a versão 2*

AULA 10 · V1

Adicionar, listar e remover



AULA 11 · HOJE

Concluídas, progresso, filtro e limpar



AULA 12

Salvar no banco de dados

ESCOPO DA V2

- ✓ Checkbox em cada tarefa para marcar e desmarcar
- ✓ Tarefa concluída aparece ~~riscada~~
- ✓ Barra de progresso: "2 de 4 concluída(s)"
- ✓ Filtro: Todas · Pendentes · Concluídas
- ✓ Botão **Limpar concluídas**

Antes de pedir, abra o projeto da aula 10 e confirme que ele roda (`streamlit run app.py`). Mudança boa começa de uma base que funciona.

Continua **sem banco**: as tarefas ainda somem ao recarregar a página. Isso é da aula 12.

Uma tarefa deixa de ser *só um texto*

Para guardar se a tarefa está feita, cada item da lista vira um **dicionário**. Essa é a mudança que puxa todas as outras:

V1 · LISTA DE TEXTOS

```
["Estudar MCP", "Revisar o PR #22"]
```

V2 · LISTA DE DICIONÁRIOS

```
[{"id": 1, "texto": "Estudar MCP", "concluida": True},  
 {"id": 2, "texto": "Revisar o PR #22", "concluida": False}]
```

POR QUE UM ID

O `id` identifica a tarefa para sempre, mesmo que ela mude de posição na lista. Vai ser a chave dos botões e checkboxes (seção 07 explica por que isso importa).

Peça ao agente para **propor o modelo de dados primeiro**. É mais fácil discutir três campos do que revisar cinquenta linhas de código.

O pedido *da versão 2*

Evolua o @app.py para marcar tarefas como concluídas:

- cada tarefa vira um dicionário com id, texto e concluida;
- um checkbox em cada tarefa marca e desmarca; concluída aparece riscada;
- barra de progresso "X de Y concluída(s)";
- filtro Todas / Pendentes / Concluídas;
- botão "Limpar concluídas".

Mantenha adicionar, remover e o aviso de tarefa vazia. Sem banco de dados. Use o id nas chaves dos widgets. No fim, rode e teste no navegador.

TRECHO	POR QUÊ
"Evolua o @app.py"	Mudança no arquivo existente, não um app novo
Lista de comportamentos	Vira o critério de pronto e o roteiro de teste
"Mantenha..."	Protege o que já funcionava
"Use o id nas chaves"	Evita o bug da seção 07 (se você já souber dele)

A rule do projeto (aula 10) já garante português, Streamlit e "sem bibliotecas novas". Não precisa repetir.

Adicionar *com id*

```
# Cada tarefa agora é um dicionário: id fixo, texto e se está concluída.
if "tarefas" not in st.session_state:
    st.session_state.tarefas = []
    st.session_state.proximo_id = 1

with st.form("nova_tarefa", clear_on_submit=True):
    col_texto, col_botao = st.columns([4, 1], vertical_alignment="bottom")
    texto = col_texto.text_input("Nova tarefa", placeholder="O que você precisa fazer?")
    adicionar = col_botao.form_submit_button("Adicionar", width="stretch")

if adicionar:
    texto = texto.strip()
    if not texto:
        st.warning("Digite uma tarefa antes de adicionar.")
    else:
        st.session_state.tarefas.append(
            {"id": st.session_state.proximo_id, "texto": texto, "concluida": False}
        )
        st.session_state.proximo_id += 1
```

- ✓ `proximo_id` é um contador guardado na sessão; cada tarefa nova pega o próximo número
- ✓ O id **nunca** é reaproveitado, mesmo depois de remover
- ✓ Toda tarefa nasce com `"concluida": False`
- ✓ O aviso de tarefa vazia e o formulário continuam iguais à v1

Ao revisar o diff, procure o que **não mudou**: se o agente mexeu em algo que já funcionava sem motivo, pergunte por quê.

Progresso, filtro *e checkboxes*

```
concluidas = sum(t["concluida"] for t in tarefas)
st.progress(concluidas / len(tarefas), text=f"{concluidas} de {len(tarefas)} concluida(s)")

filtro = st.radio("Mostrar", ["Todas", "Pendentes", "Concluídas"], horizontal=True)
visiveis = [
    t for t in tarefas
    if filtro == "Todas"
    or (filtro == "Pendentes" and not t["concluida"])
    or (filtro == "Concluídas" and t["concluida"])
]
if not visiveis:
    st.caption("Nenhuma tarefa neste filtro.")

# As chaves usam o id da tarefa, não a posição: assim, remover uma tarefa
# não faz a de baixo "herdar" o checkbox marcado.
for tarefa in visiveis:
    col_check, col_remove = st.columns([5, 1], vertical_alignment="center")
    rotulo = f"~~{tarefa['texto']}~~" if tarefa["concluida"] else tarefa["texto"]
    marcada = col_check.checkbox(rotulo, value=tarefa["concluida"], key=f"feita_{tarefa['id']}")
    if marcada != tarefa["concluida"]:
        tarefa["concluida"] = marcada
        st.rerun()
    if col_remove.button("Remover", key=f"remover_{tarefa['id']}"):
        tarefas.remove(tarefa)
        st.rerun()

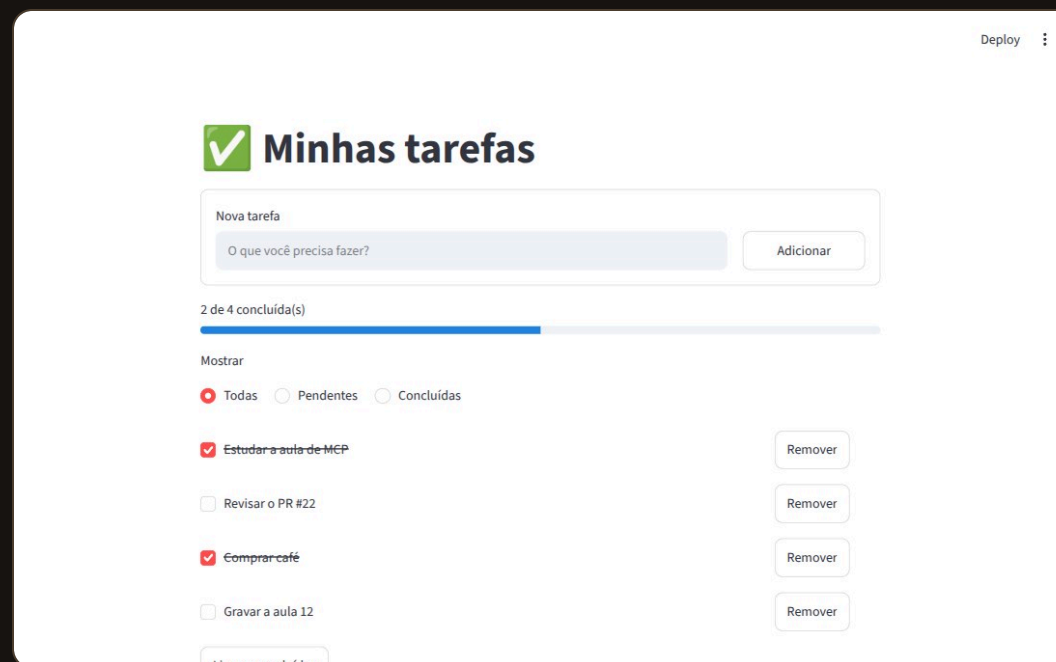
if concluidas and st.button("Limpar concluídas"):
    st.session_state.tarefas = [t for t in tarefas if not t["concluida"]]
    st.rerun()
```

sum() conta as concluídas, porque `True` vale 1.

`~~texto~~` é Markdown: o Streamlit mostra riscado.

st.rerun() redesenha na hora, com o progresso atualizado.

A versão 2 *funcionando*



Print real do `projeto/app.py` da aula, com quatro tarefas de exemplo

O bug que *quase passou*

Na primeira tentativa, o código da v2 usava a **posição** da tarefa como chave do checkbox, como a v1 fazia com o botão Remover:

```
for i, tarefa in enumerate(tarefas):  
    ... st.checkbox(..., key=f"feita_{i}")
```

- 1 Adicione A e B. Marque A como concluída.
- 2 Remova A. Agora B está na posição 0.
- 3 O Streamlit lembra que o checkbox `feita_0` estava marcado... e B aparece concluída sem ninguém ter clicado.

Por que acontece: o Streamlit guarda o estado de cada widget pela **chave**. Se a chave depende da posição, o estado "passa" para quem ocupar aquela posição.

Correção: chave pelo `id` da tarefa (`feita_{tarefa['id']}`). O id não muda quando a lista muda.

Esse bug não aparece num teste "feliz" (adicionar, marcar, ver riscado). Só aparece quando você **remove** depois de marcar. Por isso vale testar **combinações**.

Pegando bugs *com o agente*

1 · DESCREVA O SINTOMA, COM PASSOS

Bug: marquei A como concluída, removi A, e a tarefa B apareceu concluída sozinha. Passos: adicionar A e B, marcar A, remover A. Explique a causa antes de mudar qualquer coisa.

2 · PEÇA A CORREÇÃO E O TESTE

Corrija e crie um teste com o AppTest do Streamlit que reproduz esses passos, para o bug não voltar.

- ✓ **Passos para reproduzir** valem mais que "está bugado"
- ✓ "Explique antes de mudar" evita correções no chute
- ✓ Um **teste de regressão** garante que o bug não volta
- ✓ Para bugs difíceis, o modo **Debug** coleta evidência rodando o app

O código de referência desta aula passou nesse teste: adicionar A e B, marcar A, remover A → B continua **não** concluída.

Roteiro *de teste*

TESTE	ESPERADO
Marcar uma tarefa	Fica riscada; progresso sobe
Desmarcar	Volta ao normal; progresso desce
Filtro Pendentes / Concluídas	Mostra só as do grupo; "Nenhuma tarefa neste filtro" se vazio
Limpar concluídas	Somem só as marcadas
Marcar A, remover A	B continua como estava
Adicionar vazio	Aviso, como na v1

@Browser Abra o app em localhost:8501 e siga o roteiro: marque e desmarque, use os três filtros, limpe as concluídas e, por fim, marque uma tarefa, remova-a e veja se a de baixo mudou. Relate cada passo. Não altere código.

O roteiro vira o prompt de teste. É a mesma lista de comportamentos do pedido, mais as combinações que costumam quebrar.

Checklist *e commit*

- ✓ Marcar e desmarcar funcionam, com texto riscado
- ✓ Progresso correto em todas as ações
- ✓ Os três filtros funcionam
- ✓ Limpar concluídas remove só as marcadas
- ✓ Remover depois de marcar não afeta outras tarefas
- ✓ Tudo da v1 continua funcionando

```
git add app.py  
git commit -m "ToDo List: tarefas concluídas, progresso e filtro"
```

Commit **separado** da correção do bug, se ele apareceu depois: *"Corrige chaves dos checkboxes usando o id da tarefa"*. O histórico conta a história.

Teste o que *ficou*

1. Por que cada tarefa virou um dicionário?

- A) Para ocupar menos memória
- B) Para guardar id, texto e se está concluída
- C) O Streamlit exige
- D) Para salvar no banco

B. Um texto sozinho não guarda o estado.

2. Qual era a causa do bug da seção 07?

- A) O st.rerun()
- B) Chave do widget baseada na posição
- C) O filtro
- D) A barra de progresso

B. O estado ficava preso à posição.

3. Como o texto aparece riscado no Streamlit?

- A) Com CSS
- B) Com Markdown `~~texto~~`
- C) Com HTML `<s>` obrigatório
- D) Não dá

B. O rótulo do checkbox aceita Markdown.

4. Qual é a melhor forma de relatar um bug ao agente?

- A) "Está bugado"
- B) Sintoma + passos para reproduzir + "explique antes de mudar"
- C) Mandar só o print
- D) Reiniciar o Cursor

B. E pedir um teste de regressão.

Tarefas que *se completam*

Cada tarefa virou um dicionário com **id**, texto e **concluida**. Ganhamos checkbox, texto riscado, **progresso**, **filtro** e **limpar concluídas**. E aprendemos a lição do bug: chave de widget é o **id**, não a posição, e combinações de ações merecem teste.

Dicionário com id

st.checkbox · st.progress

Filtro

Chave estável

Teste de regressão

Próxima aula → 12 · Projeto ToDo List · Salvando Informações no Banco de Dados

Fontes: cursor.com/docs (Agent Overview, Debug Mode, Browser); docs.streamlit.io (st.checkbox, st.progress, st.radio, Widget keys and state, AppTest), consultados em outubro de 2026. Código de referência em `projeto/app.py`, testado com Streamlit 1.65; o bug da seção 07 foi reproduzido e corrigido nesse código.